

NEURAL NETWORK GAUSSIAN PROCESS AND CONJUGATE KERNELS

PROFESSOR ALEXANDER CLONINGER

TRISTAN ENDO



Contents

1	Introduction and Contributions	1
2	Definitions	1
2.1	Neural Networks and Infinite-Width Limits	1
2.2	Neural Network Gaussian Process (NNGP)	1
3	Closed-Form Conjugate Kernels	2
3.1	Rectified Linear Unit (ReLU) Conjugate Kernel	2
3.2	Gauss Error Function (erf) Conjugate Kernel	3
3.3	Hyperbolic Tangent (tanh) Conjugate Kernel	3
4	Kernel Recursion and Depth Dynamics	3
4.1	Kernel Recursion and Input Correlation	3
4.2	Depth Dynamics, Kernel Degeneracy, Expressivity	4
5	Experiments on Infinite-Width Kernels	4
5.1	Setup	5
5.2	Effects of Activation Functions, Weight Variance, Input Normalization	5
5.3	Finite Width Effects and the Infinite-Width Limit	6
6	Conclusion	6
7	References	6

1 Introduction and Contributions

In recent years, modern deep neural networks have been empirically successful in machine learning tasks. Still, the theoretical understanding of neural network generalization and how architectures impact function learning remains limited. One approach to analyzing these neural networks is taking the *infinite-width limit*, where randomly initialized neural networks converge to Gaussian processes. Under this approach, predictions can be described via a kernel function, determined by network architecture and activation function.

The connection between neural networks and Gaussian processes dates back to 1996, when *Radford M. Neal* proposed that single-layer neural networks with randomized weights converge to a Gaussian process when the limit of the widths is taken to ∞ . A detailed explanation can be found in his work, *Bayesian Learning for Neural Networks*. After some time, in 2018, *Jaehoon Lee* and collaborators extended this discovery to deep neural networks in their paper, *Deep Neural Networks as Gaussian Processes*. In particular, outputs of a randomly initialized deep neural network converge to a *Neural Network Gaussian Process*, or the *NNGP*, whose covariance kernel evolves recursively across network layers.

The *NNGP* provides a rigorous tool for analyzing neural network function spaces and reveals a tight relationship between neural networks and kernel methods. More specifically, the covariance kernels produced by infinite-width networks depend on the activation function chosen in addition to the parameters of the architecture: depth, weight variance, and normalized inputs. These kernels, also referred to as *conjugate kernels* in literature, can determine the class of functions that the infinite-width network can effectively represent and therefore characterize the learning bias.

In this project, we investigate the architecture and behavior of NNGP against the activation functions: *Rectified Linear Unit (ReLU)*, *Gauss Error Function (erf)*, and the *Hyperbolic Tangent (tanh)*. We will derive the corresponding closed-form conjugate kernels and examine how they evolve recursively layer by layer. Furthermore, we analyze how the depth, weight variance, and normalized inputs affect kernel degeneracy, spectral bias, and expressivity. Finally, we will compare NNGP predictions with finite-width neural networks to see when the kernel methods accurately approximate behavior and when deviations indicate feature learning beyond the conjugate kernel regime.

2 Definitions

2.1 Neural Networks and Infinite-Width Limits

Consider a fully connected *feedforward neural network* of depth L . Given an input $x \in \mathbb{R}^d$, the activations in a layer l is defined recursively by

$$g^{(l)}(y) = \phi(W^{(l)}g^{(l-1)}(y) + b^{(l)})$$

Here, $W^{(l)}$ and $b^{(l)}$ represent the weight matrix and bias vector, respectively, for layer l and $\phi(\cdot)$ is the chosen nonlinear activation function. To ensure stability per layer, weights are initialized according to a Gaussian distribution with mean 0 and variance (σ_w^2 /width of the previous layer). That is,

$$W_{ij}^{(l)} \sim \mathcal{N}\left(0, \frac{\sigma_w^2}{n_{l-1}}\right) \quad n_{l-1} = \text{width of the preceding layer}$$

Taking the limit of the width $n_l \rightarrow \infty$, each $W^{(l)}g^{(l-1)}(y) + b^{(l)}$ becomes a sum of many independent random variables. Under the *Central Limit Theorem*, each $W^{(l)}g^{(l-1)}(y) + b^{(l)}$ converges in distribution to a Gaussian random variable. As such, the neural network outputs converge to a *Gaussian Process* whose covariance function is dependent on the inputs and the network architecture.

2.2 Neural Network Gaussian Process (NNGP)

In the infinite-width limit, the covariance between the outputs corresponding to two units, y, y' is given via a kernel, defined as the following,

$$K^{(l)}(y, y') = \text{Cov}\left(g^{(l)}(y), g^{(l)}(y')\right)$$

where on a fully connected neural network, it evolves recursively across layers according to

$$K^{(l+1)}(y, y') = \sigma_b^2 + \sigma_w^2 \cdot \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Sigma)}[\phi(u)\phi(v)] \quad \text{with} \quad \Sigma = \begin{bmatrix} K^{(l)}(y, y) & K^{(l)}(y, y') \\ K^{(l)}(y', y) & K^{(l)}(y', y') \end{bmatrix}$$



This is the *Neural Network Gaussian Process Kernel*, which determines the distribution of functions on infinite-width networks. The choice of activation functions will produce different closed-form kernels, which is derived from taking the Gaussian expectation in the recursion.

The kernel describes how neural networks transform input correlations using depth to reveal properties such as expressivity and learning bias.

3 Closed-Form Conjugate Kernels

We will define the Neural Network Gaussian Process kernel recursively by taking the Gaussian expectation of the activation outputs when (u, v) , are jointly Gaussian, $\mathbb{E}[\phi(u)\phi(v)]$. The pair (u, v) has a covariance matrix determined by the kernel of the previous layer. The expectation evaluates how the input correlation changes layer by layer.

For commonly used activation functions, the expectations can be computed analytically, producing a closed-form expression of the kernel. These expressions are referred to as the *conjugate kernels*. The exact form of the conjugate kernels depends on the activation function used in the neural network, and it determines the set of functions the infinite-width network can represent. In other words, the kernel determines what kind of input-output relationships the network can learn.

To investigate the kernels, it is convenient to express the recursion in terms of the correlation between two inputs. This normalization removes the dependence on the magnitudes of the inputs and reduces the problem to a one-dimensional system, which describes how similarities between inputs evolve layer by layer. Define the kernel at layer l by $K^{(l)}(y, y')$ where y and y' are two inputs, the corresponding correlation, ρ ,

$$\rho^{(l)}(y, y') = \frac{K^{(l)}(y, y')}{\sqrt{K^{(l)}(y, y) \cdot K^{(l)}(y', y')}}.$$

The *correlation coefficient* describes how similar the representations of two inputs are at each layer of the neural network.

3.1 Rectified Linear Unit (ReLU) Conjugate Kernel

Consider the *Rectified Linear Unit (ReLU)* activation function, denoted by $\phi(z) = \max(0, z)$. When inputs, y, y' are jointly Gaussian with correlation ρ , the expectation can be analytically evaluated. Because ReLU is zero for non-positive constants, the integral is reduced to the positive quadrant.

$$\mathbb{E}[\phi(u)\phi(v)] = \int_{\mathbb{R}^2} \phi(u)\phi(v)\rho(u, v) du dv = \int_0^\infty \int_0^\infty uv\rho(u, v) du dv \quad (1)$$

The exact integral was computed by [Youngmin Cho & Lawrence Saul \(2009\)](#). They called the expectation the *arc-cosine kernel* which after evaluating the integrals, we get that if we let c be the correlation between u and v ,

$$\mathbb{E}[\phi(u)\phi(v)] = \frac{\sqrt{1-c^2} + c(\pi - \arccos(c))}{2\pi}$$

Substituting this expression back into the recursive formula produces the *ReLU kernel*, which can be written in terms of the angle between the two input vectors. The resulting *ReLU conjugate kernel* is computed in [Jaehoon Lee \(2018\)](#)

$$K^{(l+1)}(y, y') = \sigma_b^2 + \frac{\sigma_w^2}{2\pi} \sqrt{K^{(l)}(y, y)K^{(l)}(y', y')} \left(\sin \theta_{y, y'}^l + (\pi - \theta_{y, y'}^l) \cos \theta_{y, y'}^l \right)$$

$$\theta_{y, y'}^l = \arccos \left(\frac{K^{(l)}(y, y')}{\sqrt{(K^{(l)}(y, y)) \cdot (K^{(l)}(y', y'))}} \right)$$

The *ReLU conjugate kernel* is similar to that of the arc-cosine kernel and depends on the geometric relationship between the two inputs y, y' . Since ReLU is a piecewise linear function, the resulting kernel induces functions that are composed of piecewise linear transformations of the inputs.

3.2 Gauss Error Function (erf) Conjugate Kernel

Another common activation function is the *Gauss Error Function (erf)*, denoted by $\phi(z) = \text{erf}(z)$ where

$$\text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt$$

The closed-form kernel from sigmoidal activation functions was first derived by *Christopher Williams (1996)* in his work [Computing with Infinite Networks](#). Following the formulation, the expectation of the kernel can be expressed as an integral over the distribution of the network weights. Let $u \sim \mathcal{N}(0, \Sigma)$ be a random weight vector, where Σ is the covariance matrix, and let $\tilde{x} = (1, x_1, \dots, x_d)$ be an augmented input vector whose first entry corresponds to the bias. Then,

$$\mathbb{E}[\phi(x; u)\phi(x'; u)] = \frac{1}{\sqrt{(2\pi)^{d+1}|\Sigma|}} \cdot \int \phi(u^T \tilde{x})\phi(u^T \tilde{x}') \cdot \exp\left(-\frac{u^T \Sigma^{-1} u}{2}\right) du$$

Evaluating the integral analytically yields

$$\mathbb{E}[\phi(x; u)\phi(x'; u)] = \frac{2}{\pi} \arcsin\left(\frac{2\tilde{x}^T \Sigma \tilde{x}'}{\sqrt{(1 + 2\tilde{x}^T \Sigma \tilde{x})(1 + 2\tilde{x}'^T \Sigma \tilde{x}')}}\right)$$

Finally, substituting it back into the recursive formula, the closed-form expression for the *erf conjugate kernel* with two input vectors y, y' produces what is known as the *arc-sine kernel*, which shows up when Gaussian weights are combined with sigmoidal activation functions.

$$K^{(l+1)}(y, y') = \sigma_b^2 + \frac{2\sigma_w^2}{\pi} \arcsin\left(\frac{K^{(l)}(y, y')}{\sqrt{(K^{(l)}(y, y)) \cdot (K^{(l)}(y', y'))}}\right)$$

$$K^{(l)}(y, y') = 2\tilde{y}^T \Sigma^{(l-1)} \tilde{y}' \quad K^{(l)}(y, y) = 1 + 2\tilde{y}^T \Sigma^{(l-1)} \tilde{y} \quad K^{(l)}(y', y') = 1 + 2\tilde{y}'^T \Sigma^{(l-1)} \tilde{y}'$$

3.3 Hyperbolic Tangent (tanh) Conjugate Kernel

The last activation function we will investigate is the *hyperbolic tangent (tanh)*, denoted by $\phi(z) = \tanh(z)$

$$\tanh(z) = \frac{e^{2z} - 1}{e^{2z} + 1}$$

The expectation becomes $\mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Sigma)}[\tanh(u) \tanh(v)]$ with $u, v \sim \mathcal{N}(0, \Sigma)$. Plugging back into the recursive formula, we get that the *Hyperbolic Tangent conjugate kernel* is

$$K^{(l+1)}(y, y') = \sigma_b^2 + \sigma_w^2 \cdot \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Sigma)}[\tanh(u) \tanh(v)]$$

Unfortunately, the hyperbolic tangent activation function cannot produce a closed-form expression for the kernel. This is because the expectation does not produce a simple analytic closed form expression under the bivariate Gaussian distribution. For example, ReLU is piecewise linear whose integrals can be solved analytically. The erf produces a Gaussian integral, which simplifies nicely, as seen above. However, tanh is a ratio of exponentials whose integrals tend to be messy. Thus, the expectation is solved numerically (or approximated by series expansion), and in literature it is left in implicit form. The *hyperbolic tangent conjugate kernel* in recursive form remains

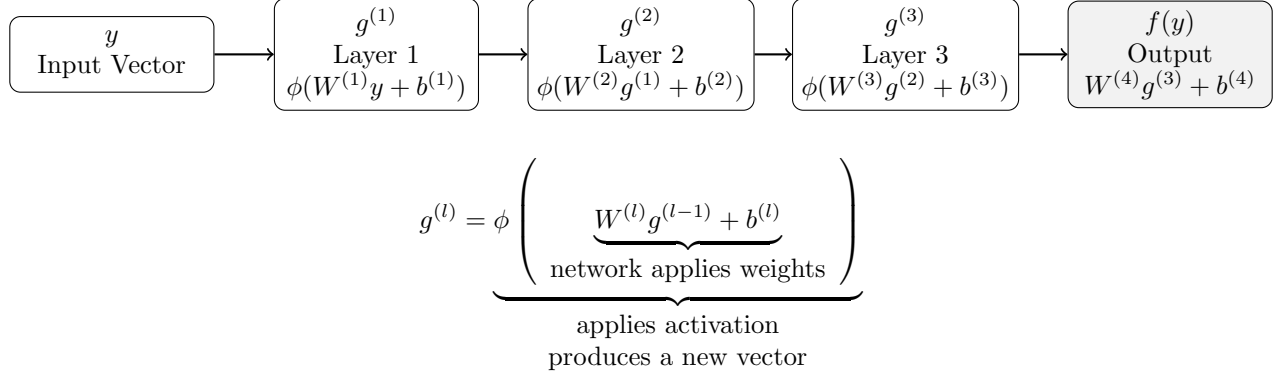
$$K^{(l+1)}(y, y') = \sigma_b^2 + \sigma_w^2 \cdot \mathbb{E}_{(u,v) \sim \mathcal{N}(0, \Sigma)}[\tanh(u) \tanh(v)]$$

where Σ is the covariance matrix of the previous layer.

4 Kernel Recursion and Depth Dynamics

4.1 Kernel Recursion and Input Correlation

The recursive kernel formulation provides a valuable understanding on how the representations evolve layer by layer. For two input vectors, $y, y' \in \mathbb{R}^d$, the kernel at layer l , $K^{(l)}(y, y')$ determines the covariance between their hidden layer activations. Recall that the recursion can be expressed in terms of the correlation between two inputs. Substituting the closed-form expectations of the examined activation functions produces a *one-dimensional recursive map*. Using $\rho^{(l)}$ to be the correlation coefficient between two inputs, $\rho^{(l)} = F(\rho^{(l-1)})$, where F depends on the activation function in addition to the weight and bias variance, σ_w^2 and σ_b^2 , respectively. This construction allows us to study the transformation of similarities between inputs as they travel through successive layers. To illustrate how representations evolve through depth, consider the following diagram of a three-layer neural network.



4.2 Depth Dynamics, Kernel Degeneracy, Expressivity

Iterating this recursion reveals that the correlation sequence converges around a *fixed point*, ρ^* , where $\rho^* = F(\rho^*)$. Once it reaches ρ^* , traversing through subsequent layers no longer affects the similarities between the inputs. In many cases, $\rho^* = 1$, meaning that as depth increases, the representations become highly correlated. The kernel also becomes nearly constant across inputs when it reaches this state. This is what is known as *kernel degeneracy*. In this regime, the network's ability to distinguish inputs has completely deteriorated since the two inputs are near identical at ρ^* .

The recursion's behavior is dependent on the activation function parameters. In particular, the weight variance, σ_w^2 , controls how the nonlinear transformation amplifies the correlation. For large σ_w^2 , correlations rapidly converge to 1, leading to kernel degeneracy. However, for small σ_w^2 , the correlation approaches 0, and the network loses useful information about input similarities. In the middle of these extremes, there is a regime where correlation is stabilized, allowing the network to keep meaningful distinctions while still applying the nonlinear transformation.

Interpreting neural networks under the infinite-width limit relies on understanding the depth dynamics. The kernel recursion shows that the activation function and the initial parameters determine how information about inputs is passed through and if architectures retain their expressivity or collapse into trivial ones. Figure 1 visualizes how the kernel recursion transforms input similarities layer by layer and how empirical fixed points arise in finite networks.

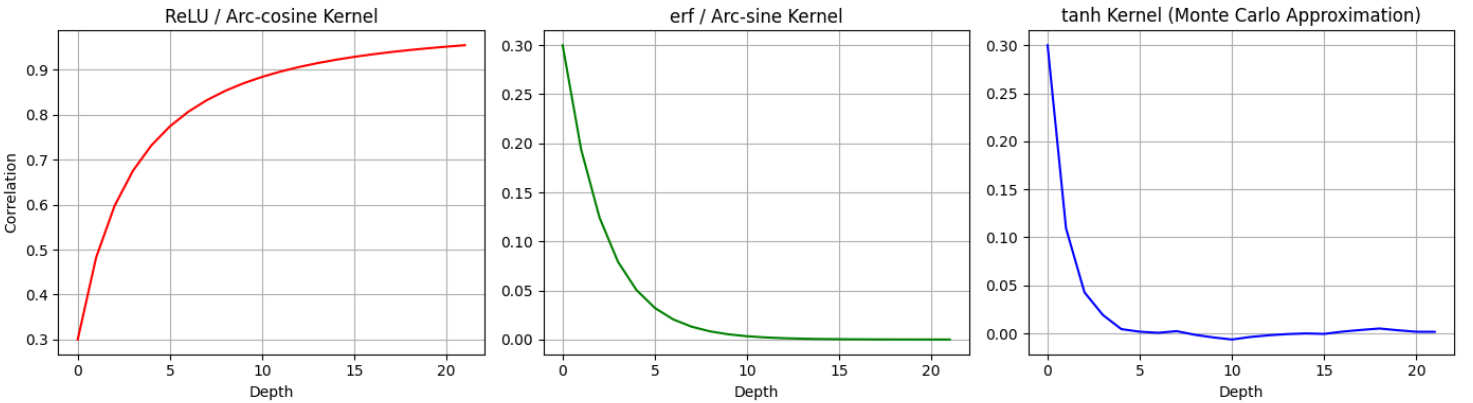


Figure 1: *Depth dynamics of input correlations under different activation functions. It highlights the evolution of $\rho^{(l)}$ between two randomly chosen inputs as a function of network depth l . Here, $\rho^{(l)}$ represents the empirical fixed point for input similarities. For ReLU networks (left), the correlation rises to 1, leading to degeneracy in finite width. For erf networks (middle) and tanh networks (right), correlations decay toward zero at finite widths due to finite-width noise.*

5 Experiments on Infinite-Width Kernels

To investigate how depth, weight variance, and finite-width effects influence kernel behavior and neural network representations, we performed experiments on fully connected networks using the ReLU, erf, and tanh activation functions. We combined the theoretical NNGP predictions with finite-width network simulations to test when infinite-width kernels can accurately describe real networks and when deviations indicate feature learning beyond the conjugate kernel regime.

5.1 Setup

To start the experiment, we take a random input vector, $y, y' \in \mathbb{R}^{21}$, a sample from the normal distribution, normalized to unit length. This simulation focuses on fully connected networks with random initial weights sampled from a Gaussian distribution with variance scaled by the input dimensions. A depth of 10-30 layers is sufficient since it is deep enough for the correlation recursion to approach the fixed point, ρ^* , and the plots can clearly show degeneracy or stabilization. So, the network was fixed to a depth of 21 layers. We tested on the ReLU, erf, and tanh activation functions, and for each, networks of width 6, 36, and 216 neurons were investigated. At each layer l , the correlation between the hidden representations of y, y' was computed by

$$\rho^{(l)} = \frac{g^{(l)}(y) \cdot g^{(l)}(y')}{\|g^{(l)}(y)\| \cdot \|g^{(l)}(y')\|}$$

where $g^{(l)}$ is the recursive activation at layer l defined in Section 2.1. This procedure was repeated through each layer to track the transformation of correlations through the network.

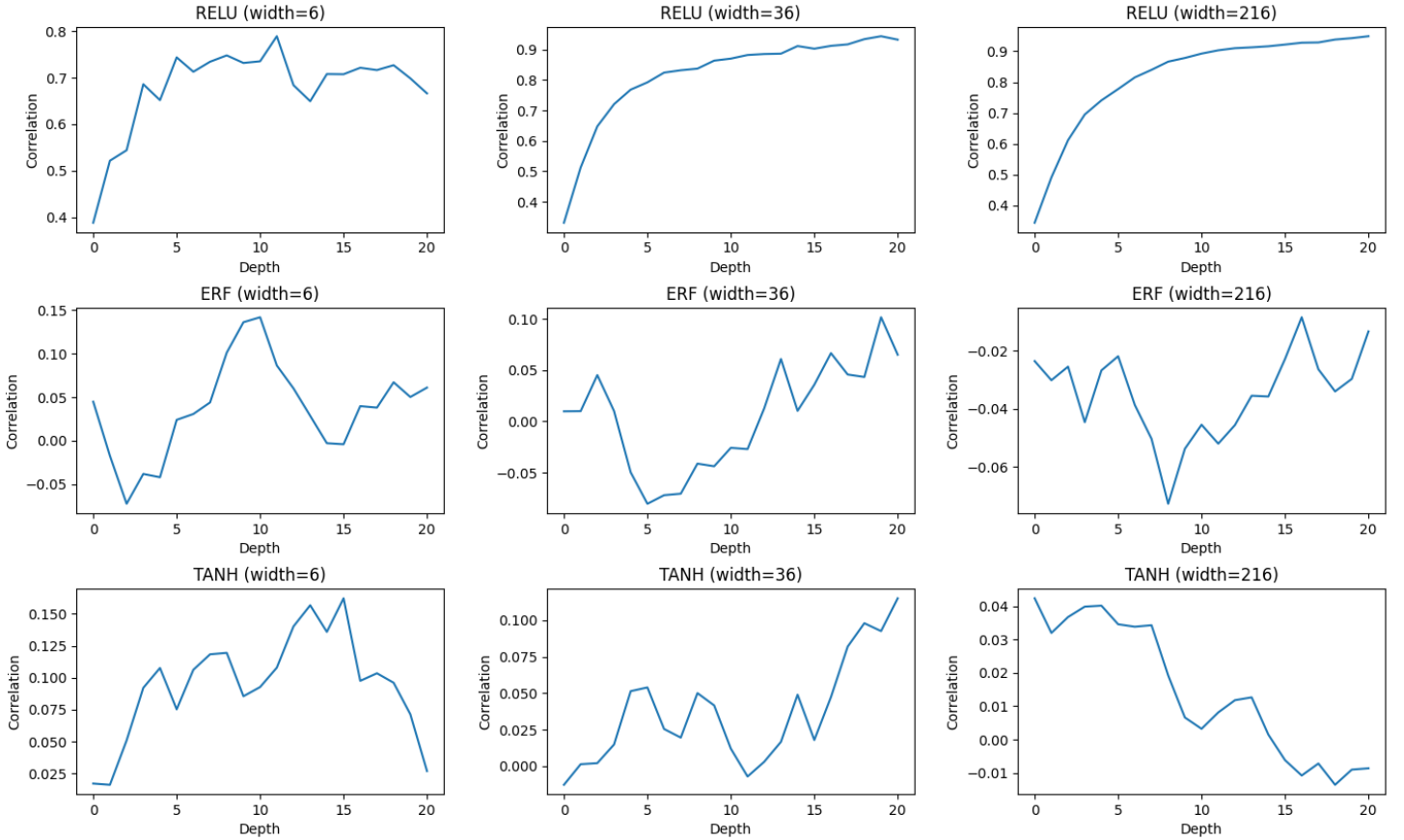


Figure 2: *Correlation dynamics for networks with widths 6, 36, 216 neurons under ReLU (left), erf (middle), and tanh (right). The correlation $\rho^{(l)}$ between two normalized inputs is tracked as a function of layer depth. As the width increases, the fluctuations from the random weights diminish, and the correlation trajectories approach the deterministic recursion predicted by the infinite-width NNGP.*

5.2 Effects of Activation Functions, Weight Variance, Input Normalization

Figure 2 displays the transformation of the correlation across depth for all three activation functions with widths 6, 36, and 216. The results tell us that the activation function has a strong influence on the transformation of input similarities.

With the ReLU activation function, the correlation coefficient is monotonically increasing toward one. In other words, the representations of the different inputs have become increasingly similar. Increasing the depth, the network under the ReLU activation function produces *kernel degeneracy*. On the contrary, the erf and tanh activation functions exhibit very different behavior. Correlations stabilize for smaller values of σ_w^2 , meaning that the activation functions tend to decorrelate the input

representations. Consequently, the network under the erf and tanh activation functions avoids kernel degeneracy and emphasizes the distinctions between two inputs. These observations illustrate how the choice of activation function influences the *expressivity* of deep neural networks through kernel recursion.

This behavior toward or away from kernel degeneracy is closely related to *spectral bias*. When the kernel reaches degeneracy, the kernel’s eigenvalues become dominated by a small number of large eigenvalues. In kernel-based models, functions associated with larger eigenvalues tend to be learned more easily than the smaller ones. When ReLU approaches degeneracy, the kernel matrix becomes nearly constant across layers, meaning the model favors smooth functions, producing a spectral bias. However, erf and tanh tend to stabilize correlations so that more features can be represented rather than being dominated. The kernel recursion tells us a great deal about how the types of activation functions can influence the class of functions the deep network can effectively represent.

In addition to activation choice, the kernel is also influenced by weight variances and input normalization. The weight variance parameter σ_w^2 controls how strongly correlations are emphasized across layers. Larger σ_w^2 values accelerate kernel degeneracy, while smaller values may suppress correlations. *Input normalization* also influences the kernel by stabilizing it, ensuring that the correlations are dependent on angular similarity for inputs. Together, these factors determine how expressive the kernel remains as depth increases and influence the neural network’s *spectral bias*.

5.3 Finite Width Effects and the Infinite-Width Limit

The experiment reveals how network widths affect the correlation trajectory. For narrow networks (width=6), the correlation exhibits significant fluctuations due to the variations in the randomly initialized weights. As the widths get larger (36 and 216), the fluctuations lessen, and the curves become smoother.

We can conclude from this trend that for wide neural networks, as the width grows, it resembles that of the recursive kernel predicted by NNGP theory. In the infinite-width limit, the kernel transformation becomes deterministic. As a result, we can say that sufficiently wide networks can approximate the infinite-width conjugate kernel behavior.

6 Conclusion

While the experiments investigate randomly initialized networks, they allow us to visualize how neural networks behave under the conjugate kernel. In the infinite-width limit, training a neural network with *gradient descent* can be described as kernel regression with the *Neural Tangent Kernel (NTK)*, which governs the evolution of predictions during optimization. In this setting, the network learns with a fixed kernel that depends on the architecture and the activation function.

On the other hand, finite-width neural networks may deviate from this kernel regime during training. With limited width, the network parameters could alter the effective feature representation, allowing the network to learn features beyond those captured by the initial conjugate kernel. From experimentation, for neural networks to behave like the kernel, the width must be sufficiently large enough. Moreover, it emphasized how finite-width networks could deviate from this behavior through feature learning.

7 References

- Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. *Deep Neural Networks as Gaussian Processes*. pp. 3-6, 12-13, 2018
- Christopher K.I. Williams. Computing with Infinite Networks. *In Advances in Neural Information Processing Systems*. pp. 295-301, 1997
- Youngmin Cho and Lawrence K. Saul. Kernel methods for deep learning. *In Advances in Neural Information Processing Systems*, pp. 342–350, 2009
- Radford M. Neal. *Bayesian Learning for Neural Networks*. PhD thesis, University of Toronto, Dept. of Computer Science, 1994b